

CareHandoff AI: Multi-Strategy RAG System for Discharge Note Auditing

<https://carehandoff-ai.onrender.com/>

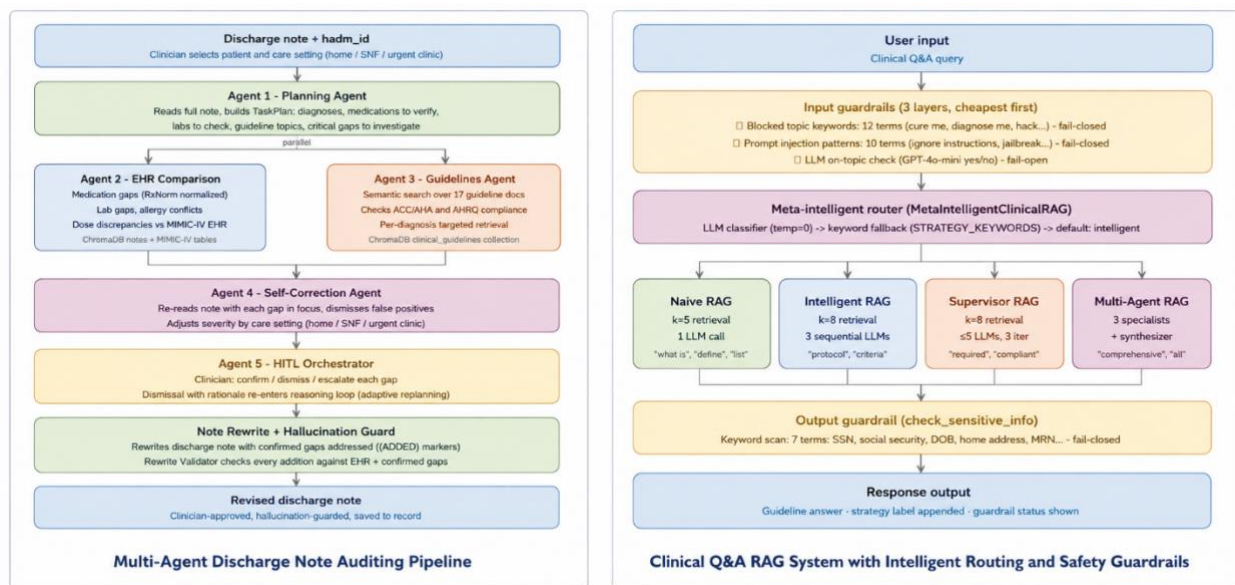
Section 1: Executive Summary

CareHandoff AI is a dual-component clinical decision-support platform for hospital discharge safety. The primary component is a five-agent discharge note auditor: a Planning Agent decomposes the note before any retrieval, parallel EHR Comparison and Guidelines agents cross-reference the note against MIMIC-IV patient records and 17 clinical guidelines, a Self-Correction Agent dismisses false positives and adjusts severity by care setting, and a Human-in-the-Loop Orchestrator routes every verified gap to a clinician for approval before any action is taken. The second component is a Clinical Q&A interface powered by MetaIntelligentClinicalRAG: a router that classifies each query and dispatches it to one of four RAG strategies (Naive, Intelligent, Supervisor, or Multi-Agent) based on complexity. The Q&A interface is wrapped in GuardedClinicalRAG, which enforces three input guardrails and one output guardrail. The knowledge base spans 19 guideline files (17 clinical domains: ACC/AHA, AHRQ, ACG, ADA) embedded in ChromaDB, plus structured MIMIC-IV patient data. The system serves physicians and discharge coordinators who need accurate answers to documentation standards questions, protected against personal medical advice requests, prompt injection, and sensitive data leakage.

Section 2: Knowledge Base Description

- **Domain:** Hospital discharge documentation standards across 17 conditions, chosen because discharge-to-handoff communication failures are linked to 70% of adverse events at care transitions, and discharge notes are free-text, highly variable in quality, and the primary vehicle for communicating care plans.
- **Guideline corpus (17 domains, 19 files):** All .txt files in data/raw/guidelines/: ACC/AHA for Heart Failure, Hypertension, Atrial Fibrillation, Chronic Coronary Disease, CAD, Coronary Revascularization, PAD, Arrhythmias, Valvular Heart Disease, Perioperative Cardiovascular Management; ADA Diabetes standards (3 files); GOLD COPD; ACG Hepatic Encephalopathy; AHRQ Pneumonia; AHRQ RED Toolkit; AHA/ASA Stroke and TIA; and AKI/AKD guidelines.
- **Structured EHR data (MIMIC-IV):** discharge.csv (free-text notes) plus 8 .csv.gz structured tables: prescriptions, labevents, d_labitems, diagnoses_icd, d_icd_diagnoses, procedures_icd, d_icd_procedures, admissions. Drug names are normalized via the NLM RxNorm REST API before EHR comparison so "Metoprolol Succinate ER" and "metoprolol succinate" resolve to the same RxCUI.
- **Chunking and embedding:** Notes: section-aware split on 16 MIMIC section headers (e.g. "Discharge Medications:", "Pertinent Results:"), then RecursiveCharacterTextSplitter (chunk_size=1200, overlap=100) for oversized sections. Guidelines: RecursiveCharacterTextSplitter (chunk_size=3000, overlap=150). Deterministic SHA-256 chunk IDs enable idempotent re-ingestion. Embeddings use text-embedding-3-small routed explicitly to the standard OpenAI API (OpenRouter excluded). Two ChromaDB collections: "discharge_notes" and "clinical_guidelines". Relevance threshold: 0.3.

Section 3: Architecture Diagram



Section 4: Routing Logic

- Step 1 - LLM classifier:** GPT-4o-mini (temperature=0) receives the query and returns exactly one of four strategy names. The system prompt instructs the model to respond with only the label; if the output isn't a valid label, the classifier is treated as failed and keyword fallback runs.
- Step 2 - Keyword fallback:** STRATEGY_KEYWORDS iterated in insertion order (naive → intelligent → supervisor → multi_agent); first match wins. **Default:** "intelligent" if no match.

Query Example	Detected Pattern	Routes To
"What is the AHRQ Re-Engineered Discharge Toolkit?"	LLM: factual definition. Fallback: "what is" keyword.	Naive RAG
"What are the steps for documenting heart failure discharge?"	LLM: protocol/steps. Fallback: "steps for" keyword.	Intelligent RAG
"Is it required to document pending labs in the discharge note?"	LLM: compliance check. Fallback: "is it required" keyword.	Supervisor RAG
"Give me a comprehensive overview of all discharge documentation requirements."	LLM: multi-area. Fallback: "comprehensive" + "all requirements".	Multi-Agent RAG

Section 5: Guardrails Implementation

Execution order (cheapest first): Blocked topic keywords → prompt injection → LLM on-topic check → RAG → sensitive field scan. The two keyword input guardrails cost zero LLM tokens; the LLM check only fires after both pass.

Fail-open rationale: The LLM on-topic check explicitly catches exceptions and returns {"allowed": True} so a network error never blocks a clinician from a legitimate documentation query. All other guardrails are fail-closed.

Guardrail	Type	Threat Blocked	Mechanism & Fail Behavior
-----------	------	----------------	---------------------------

Blocked Topic Keywords	Input	Personal medical advice: "cure me", "diagnose me", "prescribe me", "my medication"; illegal: "hack", "exploit", "weapon"	Keyword match: 12 blocked_topics strings. Fail-closed.
Prompt Injection	Input	Jailbreak / override: "ignore previous instructions", "act as", "jailbreak", "system prompt", "pretend to be"	Keyword match: 10 injection patterns. Fail-closed.
LLM On-Topic Check	Input	Off-topic or personal advice not caught by keywords (e.g. "how do I lower my blood pressure")	GPT-4o-mini binary yes/no. Fail-OPEN: exceptions pass through to protect availability.
Sensitive Info Scan	Output	PII in response: "ssn", "social security", "home address", "personal phone", "date of birth", "mrn", "medical record number"	Keyword match: 7 sensitive_fields strings on response text. Fail-closed.

Section 6: Tradeoffs and Design Decisions

Why not use Multi-Agent for every query?

Multi-Agent makes 4 LLM calls vs Naive's 1. More critically, specialists are hardcoded to Heart_Failure.txt, Hypertension.txt, and redtoolkit.txt, for 14 of 17 guideline domains, the "specialist" is just the general toolkit, making Multi-Agent worse than Intelligent RAG's k=8 broad retrieval for most conditions.

Why not use Supervisor for every query?

Supervisor runs up to 3 generate → evaluate → refine iterations (max 5 LLM calls). For simple lookups the first generation is already correct; the supervisor evaluate call returns "SUFFICIENT" immediately but still costs an LLM call with no gain.

Why use a cheaper model for guardrails?

The guardrail LLM uses the same model (GPT-4o-mini, temperature=0) but outputs only "yes" or "no", minimal tokens. The two keyword guardrails cost zero API tokens; the LLM check only fires after both passes, so most blocked queries incur no API cost.

What happens if the router makes a wrong decision?

LLM failure triggers keyword fallback; no match defaults to "intelligent". Critical failure: STRATEGY_KEYWORDS is iterated naive-first. "What are all requirements for discharge?" matches "what are" (naive) before "all requirements" (multi_agent), silently under-serving the user with 1 LLM call instead of 4.

Section 7: Limitations and Future Work

Limitations

- Multi-Agent hardcoded to 3 specialists (Heart_Failure.txt, Hypertension.txt, redtoolkit.txt). Queries about AKI, AFib, COPD, Stroke, Diabetes, or 12 other domains fall through to the general guidelines specialist, making Multi-Agent no better than Intelligent RAG for most of the knowledge base.
- No conversation memory: chat() receives Gradio's history list but none of the four strategy functions include it in the LLM prompt. Follow-up queries like "what about for hypertension specifically?" fail because there is no prior context in the generation call.
- Keyword fallback is order-sensitive: STRATEGY_KEYWORDS is a plain dict iterated in insertion order (naive first). A query with overlapping signals matches the first strategy, not the most appropriate one, with no notification to the user.

Future Improvements

- Dynamic specialist routing: Parse condition terms from the query at runtime and map them to the appropriate `source_filter` from all 17 guideline files, rather than hardcoding 3 specialists.
- Conversation history injection: Pass the last 3 turns from Gradio's history list into each strategy function's LLM prompt so follow-up questions resolve without context repetition.